# LLBox × X360 Integration

Flow reference, gotchas, and working implementation

Merged from the LLBox X360 Integration Reference (API v1) and the bridge service built on top of it.

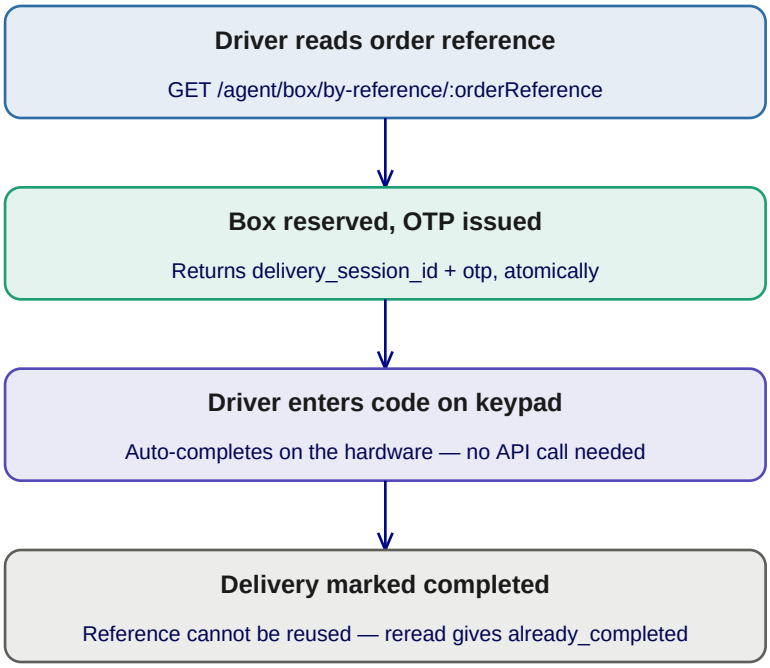
# Overview

LLBox owns the parcel-box hardware and OTP generation. X360's job is purely the calling layer: resolve a delivery to a box, hand the driver a code, and (usually) let the hardware close it out. Everything is server-to-server, JSON in/out, single header auth (`X-Agent-API-Key`). There are two independent entry points into the system — they don't have to be used together.

Base URL: `https://llbox.up.railway.app/api/v1`
Auth: single `X-Agent-API-Key` header, no expiry, revoked only on manual rotation
Environment: production only — no sandbox. Test calls are real traffic, real rate limits.

## Journey 1 — Driver verification

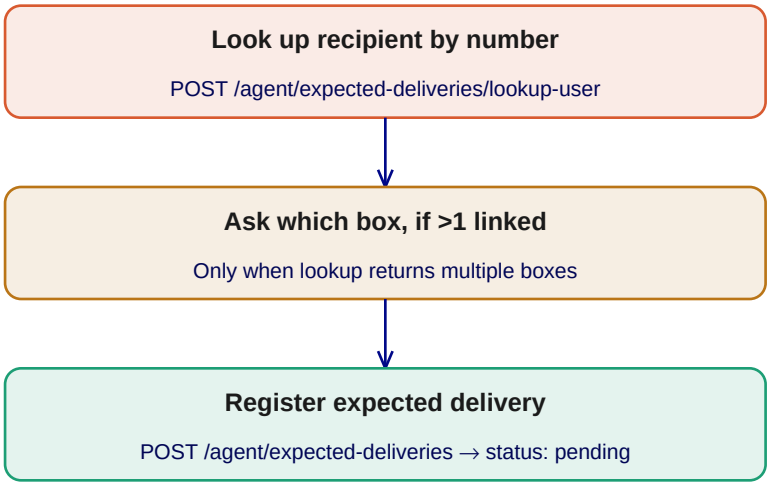
The end-to-end flow when a driver is standing at the box with an order reference already registered.




**Fallback:** if the box is offline or the driver confirms verbally instead of via keypad, call `POST /agent/delivery/confirm` with the `delivery_session_id` from step 1 to close it out manually — same end state as step 4.

## Journey 2 — WhatsApp registration

Used when the agent identifies a recipient over WhatsApp and registers a delivery on their behalf, before or independent of a driver ever calling in. The reference registered here is what feeds Journey 1.




Resubmitting the same reference for the same account is idempotent — returns the original `201` again rather than erroring, so retries on your side are safe.

# Status values

| Box status | Meaning |
|---|---|
| available | Free to claim |
| in_use | Reserved for an in-progress delivery — other claims 409 until it clears |
| maintenance / deactivated | Offline — claims rejected with 403 |

| Expected delivery status | Meaning |
|---|---|
| pending | Waiting for a driver — only state by-reference will claim |
| matched | Driver resolved it, not yet confirmed complete |
| completed | Permanent — reference cannot be reused |
| cancelled / expired | Cancelled before claim / hold window passed |

# Gotchas to handle before shipping

### otp/validate always returns HTTP 200

The real result is in `data.valid`. If your error middleware branches on status code, a failed check silently reads as success.

### No sandbox environment

Every test call is production traffic under production rate limits. Use a clearly fake, greppable reference for connectivity checks, e.g. `X360-TEST-<date>`.

### Rate limit is per source IP, not per API key

30 req / 5 min on `by-reference`, 100 req / 15 min overall. If this runs behind a shared proxy or NAT, that budget is shared across everything on the IP.

### 409 ALREADY_CONFIRMED on manual confirm is not an error

It means the delivery already completed. Blind retry-on-error logic will loop on this.

### Box claim races are real

A 409 on `by-reference` means another session grabbed the box a moment earlier — this needs a retry/backoff path, not a hard failure.

# Appendix — implementation

Bridge service exposing both journeys as plain HTTP tool endpoints for an agent runtime (LangGraph tool, n8n HTTP node, etc.) to call.

### *llboxClient.js — low-level API wrapper*

```
const BASE_URL = process.env.LLBOX_BASE_URL || "https://llbox.up.railway.app/api/v1";
const API_KEY = process.env.LLBOX_API_KEY;

if (!API_KEY) {
  throw new Error("LLBOX_API_KEY is not set. Export it before starting the service.");
}

/**
 * Low-level request helper.
 * - Retries once on 500 (per spec: "safe to retry once").
 * - Never throws on 4xx — those are meaningful business responses (not_found,
 *   already_completed, box_selection_required, etc.), and the caller needs the
 *   body + status to branch correctly. Only network failures / non-JSON bodies throw.
 */
async function request(method, path, { body, retried = false } = {}) {
  const res = await fetch(`${BASE_URL}${path}`, {
    method,
    headers: {
      "X-Agent-API-Key": API_KEY,
      ...(body ? { "Content-Type": "application/json" } : {}),
    },
    body: body ? JSON.stringify(body) : undefined,
  });

  let data;
  try {
    data = await res.json();
  } catch {
    throw new Error(`LLBox returned a non-JSON response (${res.status}) for ${method} ${path}`);
  }

  if (res.status === 500 && !retried) {
    return request(method, path, { body, retried: true });
  }

  return { status: res.status, data };
}

// --- Driver verification --------------------------------------------------

/**
 * Resolves an order reference to a box + fresh OTP, atomically reserving it.
 * Rate-limited to 30 req / 5 min per IP — don't call this speculatively.
 */
async function resolveBoxByReference(orderReference) {
  return request("GET", `/agent/box/by-reference/${encodeURIComponent(orderReference)}`);
}

/** Admin/testing fallback — resolves by box code or numeric ID directly. */
async function resolveBoxByIdentifier(identifier) {
  return request("GET", `/agent/box/${encodeURIComponent(identifier)}`);
}

/**
 * Manual confirmation fallback. Only call this when the box can't take the
 * keypad code itself (offline) or the agent confirmed delivery verbally.
 * A 409 ALREADY_CONFIRMED means the delivery already completed — treat as
 * success, not a failure, when deciding how to respond to the caller.
 */
async function confirmDeliveryManually(payload) {
  const result = await request("POST", "/agent/delivery/confirm", { body: payload });
  if (result.status === 409 && result.data.code === "ALREADY_CONFIRMED") {
```

```
    return { ...result, alreadyConfirmed: true };
  }
  return result;
}

/**
 * Always returns HTTP 200 — the real answer is in data.valid.
 * Do not treat this call's status code as a success/failure signal.
 */
async function validateOtp(payload) {
  const { data } = await request("POST", "/agent/otp/validate", { body: payload });
  return data; // { valid, reason, expires_at? }
}

async function resetBox(boxId, resetBy) {
  return request("POST", `/agent/box/${boxId}/reset`, { body: resetBy ? { reset_by: resetBy } : {} });
}

// --- WhatsApp registration ------------------------------------------------

async function lookupUserByWhatsapp(whatsappPhone) {
  return request("POST", "/agent/expected-deliveries/lookup-user", {
    body: { whatsapp_phone: whatsappPhone },
  });
}

async function registerExpectedDelivery({ whatsappPhone, orderReference, boxId }) {
  return request("POST", "/agent/expected-deliveries", {
    body: {
      whatsapp_phone: whatsappPhone,
      order_reference: orderReference,
      ...(boxId ? { box_id: boxId } : {}),
    },
  });
}

module.exports = {
  resolveBoxByReference,
  resolveBoxByIdentifier,
  confirmDeliveryManually,
  validateOtp,
  resetBox,
  lookupUserByWhatsapp,
  registerExpectedDelivery,
};
```

## *journeys.js — journey orchestration*

```
const llbox = require("./llboxClient");

/**
 * JOURNEY 1 — Driver verification
 * Driver is standing at the box with an order reference. Resolve it to a box
 * + OTP, hand the code to the driver. Confirmation normally happens on the
 * hardware automatically; only call confirmDelivery() if the box is offline
 * or the agent is confirming verbally.
 */
async function handleDriverVerification(orderReference) {
  const { status, data } = await llbox.resolveBoxByReference(orderReference);

  if (status === 200) {
    return {
      outcome: "resolved",
      say: `Deliver to ${data.address}. Enter code ${data.otp} on the box keypad.`,
      box: {
        boxId: data.box_id,
        address: data.address,
        mapsLink: data.google_maps_link,
        otp: data.otp,
        otpExpiresAt: data.otp_expires_at,
        deliverySessionId: data.delivery_session_id,
      },
    };
  }

  if (status === 404) {
    // reason is one of: not_found, expired, already_matched, already_completed, cancelled
    const messages = {
      not_found: "I don't have a delivery registered against that reference.",
      expired: "That reference's hold window has passed.",
      already_completed: "That delivery is already marked complete.",
      already_matched: "That reference is already being handled by another session.",
      cancelled: "That delivery was cancelled before pickup.",
    };
    return { outcome: "not_claimable", reason: data.reason, say: messages[data.reason] || data.message };
  }

  if (status === 403) {
    // reason: maintenance | deactivated
    return { outcome: "box_unavailable", reason: data.reason, say: "That box is temporarily offline — please try
  }

  if (status === 409) {
    return { outcome: "conflict", say: "Another delivery may be in progress for that box right now — please retry
  }

  return { outcome: "error", status, say: "Something went wrong on our end — please try again." };
}

/**
 * Optional manual close-out when the box can't take the keypad code itself.
 */
async function confirmDeliveryManually({ deliverySessionId, boxId, driverPhone, otpProvided, parcelCount, vendorN
  const { status, data, alreadyConfirmed } = await llbox.confirmDeliveryManually({
    delivery_session_id: deliverySessionId,
    box_id: boxId,
    driver_phone: driverPhone,
    otp_provided: otpProvided,
    parcel_count: parcelCount,
    vendor_name: vendorName,
    notes,
    confirmed_by: "agent",
  });

  if (status === 200 || alreadyConfirmed) {
    return { outcome: "completed", say: "Delivery confirmed." };
  }
  if (status === 401) {
```

```
    return { outcome: "otp_mismatch", say: "That code doesn't match — please double-check with the driver." };
  }
  return { outcome: "error", status, say: data.message || "Couldn't confirm the delivery." };
}

/**
 * JOURNEY 2 — WhatsApp registration
 * Agent has identified a recipient over WhatsApp and needs to register their
 * expected delivery ahead of the driver showing up. Handles the multi-box
 * disambiguation step inline: if boxId isn't supplied and the account has
 * more than one box, returns outcome "choose_box" with the options so the
 * caller can ask the recipient and retry with boxId set.
 */
async function handleWhatsappRegistration({ whatsappPhone, orderReference, boxId }) {
  const lookup = await llbox.lookupUserByWhatsapp(whatsappPhone);

  if (lookup.status === 404 || lookup.data.user_found === false) {
    return { outcome: "no_account", say: "I don't see an LLBox account linked to that number." };
  }

  if (!boxId && lookup.data.boxes.length > 1) {
    return {
      outcome: "choose_box",
      say: "This account has more than one box — which one is this delivery for?",
      boxes: lookup.data.boxes,
    };
  }

  const resolvedBoxId = boxId || lookup.data.boxes[0]?.box_id;
  const { status, data } = await llbox.registerExpectedDelivery({ whatsappPhone, orderReference, boxId: resolvedB

  if (status === 201) {
    return { outcome: "registered", say: `Delivery registered for ${data.box_code}. Reference is ready for the dr
  }
  if (status === 422 && data.code === "BOX_SELECTION_REQUIRED") {
    return { outcome: "choose_box", say: "Which box is this delivery for?", boxes: data.boxes };
  }
  if (status === 422 && data.code === "NO_BOX_LINKED") {
    return { outcome: "no_box", say: "That account doesn't have a box linked yet." };
  }
  if (status === 403 && data.code === "BOX_NOT_OWNED") {
    return { outcome: "wrong_box", say: "That box isn't linked to this account." };
  }
  if (status === 409 && data.code === "REFERENCE_ALREADY_ACTIVE") {
    return { outcome: "already_active", say: "That reference is already registered on another account." };
  }
  return { outcome: "error", status, say: data.message || "Couldn't register the delivery." };
}

module.exports = { handleDriverVerification, confirmDeliveryManually, handleWhatsappRegistration };
```

## *server.js — HTTP tool endpoints*

```
const express = require("express");
const { handleDriverVerification, confirmDeliveryManually, handleWhatsappRegistration } = require("./journeys");

const app = express();
app.use(express.json());

// Journey 1 — driver at the box, reading out a reference
app.post("/tools/resolve-delivery", async (req, res) => {
  const { orderReference } = req.body;
  if (!orderReference) return res.status(400).json({ error: "orderReference is required" });
  res.json(await handleDriverVerification(orderReference));
});

// Journey 1 (optional step) — manual confirmation when the box can't take the code
app.post("/tools/confirm-delivery", async (req, res) => {
  const { deliverySessionId, boxId } = req.body;
  if (!deliverySessionId || !boxId) return res.status(400).json({ error: "deliverySessionId and boxId are require
  res.json(await confirmDeliveryManually(req.body));
});

// Journey 2 — register a delivery from a WhatsApp conversation
app.post("/tools/register-delivery", async (req, res) => {
  const { whatsappPhone, orderReference } = req.body;
  if (!whatsappPhone || !orderReference) return res.status(400).json({ error: "whatsappPhone and orderReference a
  res.json(await handleWhatsappRegistration(req.body));
});

app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({ error: "internal error" });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`LLBox X360 bridge listening on :${PORT}`));
```

## *package.json*

```
{
  "name": "llbox-x360-integration",
  "version": "1.0.0",
  "description": "X360 <-> LLBox server-to-server bridge",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "engines": {
    "node": ">=18"
  },
  "dependencies": {
    "express": "^4.19.2"
  }
}
```

## *README.md — setup & test commands*

````
# LLBox X360 bridge

Wraps the LLBox API for X360's voice/WhatsApp agent. Exposes two journeys as
plain HTTP tool endpoints so they can be wired into a LangGraph tool, an n8n
HTTP Request node, or whatever the agent runtime is.

## Setup

```bash
npm install
export LLBOX_API_KEY="600dcc8975de32a58eda98e4bfc262e85452ea935b7af2c20db162ab31d1df2b"
npm start
```

No sandbox exists on LLBox's side — every call, including these tests, hits
production. Use a clearly fake reference for connectivity checks, e.g.
`X360-TEST-20260802`.

## Journey 1 — driver verification

```bash
curl -X POST localhost:3000/tools/resolve-delivery \
  -H "Content-Type: application/json" \
  -d '{"orderReference": "X360-TEST-20260802"}'
```

`outcome` will be one of: `resolved`, `not_claimable`, `box_unavailable`,
`conflict`, `error`. Only `resolved` carries an OTP — read `box.otp` to the
driver, don't call anything else. The box's own keypad completes the
delivery automatically.

If the box can't take the code (offline) or the agent confirms verbally:

```bash
curl -X POST localhost:3000/tools/confirm-delivery \
  -H "Content-Type: application/json" \
  -d '{"deliverySessionId": "42", "boxId": 7}'
```

## Journey 2 — WhatsApp registration

```bash
curl -X POST localhost:3000/tools/register-delivery \
  -H "Content-Type: application/json" \
  -d '{"whatsappPhone": "+966500000000", "orderReference": "ORDER-99231"}'
```

If `outcome` is `choose_box`, ask the recipient which box, then retry with
`boxId` set to their answer.

## Gotchas already handled in the client

- `/agent/otp/validate` always returns HTTP 200 — `journeys.js` reads
  `data.valid`, never the status code, if you extend it to use that endpoint.
- 500s are retried once automatically (`llboxClient.js`), per spec.
- 409 `ALREADY_CONFIRMED` on manual confirm is treated as success, not an
  error — the delivery is done either way.
- Rate limits are per source IP, not per key — if this runs behind a shared
  proxy/NAT, that 30-req/5-min budget on `by-reference` is shared across
  everything on that IP.
````